

발표 요약

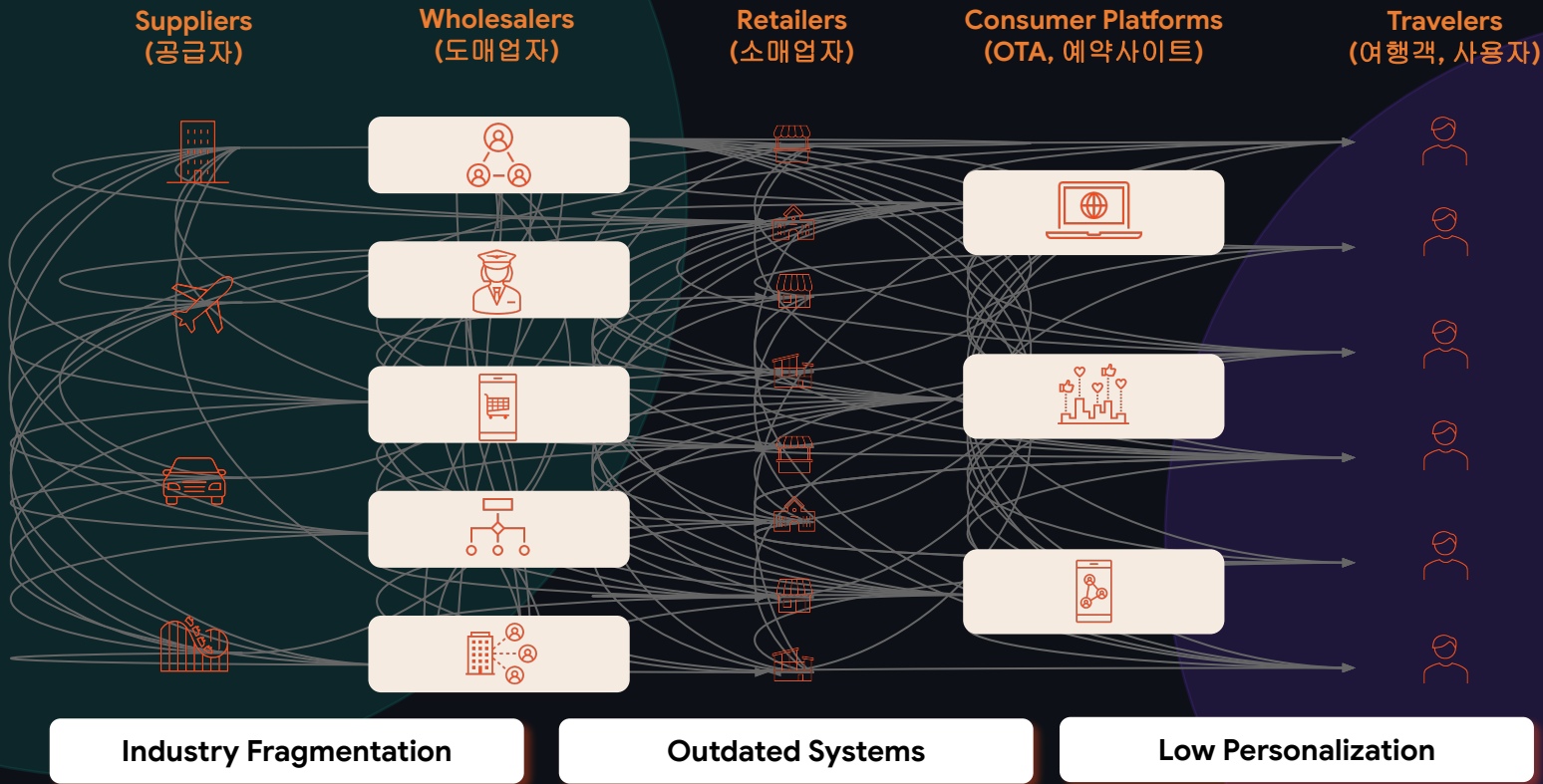
기업에서 **AI 전환(AX)**이 내부 생산성과 외부 서비스 혁신이라는 두 축으로 실제로 어떻게 구현되고 있는지를 보여줍니다. 이를 바탕으로 **AI**가 서비스 개발의 진입 장벽은 낮췄지만 아키텍처·보안·품질 판단 같은 역량의 천장은 그대로이며, 특히 정답이 없는 영역에서는 사람의 판단이 필수적인 영구적 아키텍처로 남는다는 통찰을 전달합니다. 마지막으로 **Demo**와 실제 **Product** 사이의 큰 간극을 짚으며, **CS** 학생·연구자들에게는 깊이·넓이·연결을 갖춘 **T**자형 인재로 성장할 것을 제안하는 실무 중심의 강연입니다.

기업에서 바라보는 AX와 서비스

7/13/2026

이준영 (Yanolja)

Yanolja는 한국 및 글로벌을 대상으로 여행, 여가 및 문화 산업의 전반에 걸쳐 B2B와 B2C 서비스를 제공하고 있습니다.



Yanolja는 엔터프라이즈 솔루션(Enterprise Solutions)과 컨슈머 플랫폼(Consumer Platform)을 통해 생산성과 개인화를 한층 높이며, 여행 산업의 새로운 혁신을 선도하고 있습니다.

Yanolja Enterprise Solutions (B2B)



Hospitality Solutions

호텔 운영 환경을 디지털 전환하여 데이터의 효율적인 활용을 통해 그들에게 비즈니스 솔루션을 제공



Transaction Solutions

전 세계 200개국 이상의 여행 서비스 공급자와 온라인·오프라인 판매 채널을 연결



Data Solutions

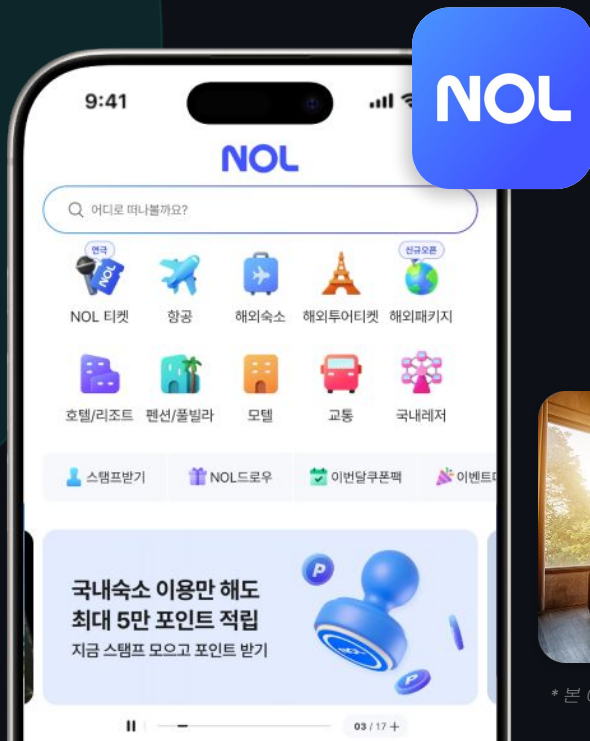
독자적인 여행 특화 데이터와 AI 기반 기술을 활용한 분석·해석 및 의사결정 고도화



NOL UNIVERSE

여행, 레저, 문화 플랫폼으로, '휴식과 행복'의 여정을 더욱 풍요롭게 만드는 다양한 서비스를 제공

놀유니버스는 숙박, 항공은 물론 패키지 투어·교통·공연·전시·스포츠까지
삶을 풍요롭게 만드는 다양한 즐거움을 제공합니다.



* 본 이미지는 이해를 돕기 위한 예시로, 실제와 다를 수 있습니다.

Yanolja's Global Presence

50+

글로벌 오피스

200+

글로벌 서비스 국가



기업에서 바라보는 AI - 아직은 기대반 걱정반

이젠 AI로 뭐든 다 할 수 있어
(요술방망이)!

그런데, AI로 돈을 어떻게 벌지?

Token 비용이 엄청나다고 하던데?

누구나 다 코딩을 할 수 있다고?

AI가 다하니까 직원을 줄여!

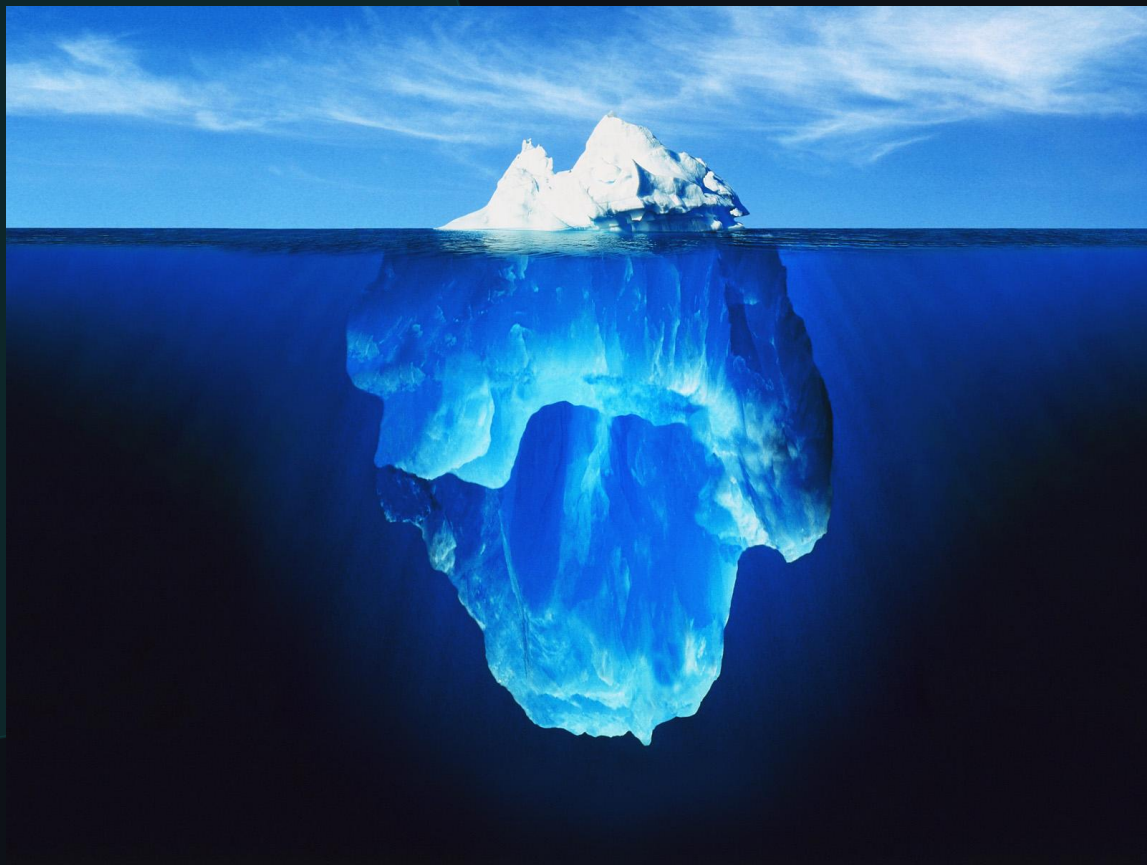
AI기업들은 단기간에 1조매출을
올린다던데

보안 위험은 더 커진다던데?

그런데 우리는 뭘, 어떻게 하지?

AI로 제품 개발을 몇 일만에 한다던데?

AX에 대한 냉정한 현실



기업의 AX는 두 가지 Track으로 진행됩니다



내부 생산성 (Productivity)

코딩, 문서, 협업, 마케팅
같은 인원으로 더 많이, 더 빠르게

VS



외부 서비스 (Product)

새로운 경험의 서비스 보조 수단, 산업 재정의
AI가 핵심 가치가 되는 서비스, 경쟁 우위의 재정의

두 Track은 독립적으로 추진되지만, 상호 학습하며 함께 성숙해갑니다

AI가 가장 큰 부분을 차지하는 영역 - 코딩

비개발자와 개발자가 바라보는 서로 다른 세계 — 개발도, AI도 마찬가지입니다

👁 비개발자 또는 관리자가 보는 것

완성된 UI / 동작하는 Demo

"기능이 되네요!" "언제 출시해요?"

수면 위 — 보이는 세계

🏠 개발자가 만드는 것 (수면 위)

UI / Demo / MVP

"사용자에게 보여지는 것"

수면 위 — 시작점

— 수면 (Surface) —

보이지 않는 영역
(인식 밖)

?

?

?

🏗 개발자가 만드는 것 (수면 아래)

⚙ 아키텍처 설계 / 시스템 구조

🔒 보안 · 인증 · 인가

📈 스케일링 · 가용성 · SLA

🔍 Harness · Eval · QA · 회귀 테스트

🔄 운영 · 모니터링 · 장애 대응

📊 비용 최적화 · 캐싱 · 성능

🌐 Localization · 접근성 · 법적 요건

수면 위를 띄우기 위해, 수면 아래 90%의 빙산을 튼튼하게 만드는 데 대부분의 노력을 기울입니다

뛰어난 개발자에게 더 많은 기회가 있다

같은 AI를 쓰지만 — 벽을 넘느냐, 못 넘느냐의 차이

①



에러를 읽는 능력

비개발자

AI한테 다시 부탁할 뿐

개발자

계층별 진단 — 프롬프트? 로직? 인프라?

복잡도가 올라갈수록 이 차이가 폭발적으로 벌어짐

②



추상화 레벨 선택

비개발자

프롬프트 레이어에만 갇힘

개발자

Prompt / Code / Infra 중 최적 레이어 선택

No-Code 툴이 지원하지 않는 것은 할 수 없음

③



Eval 설계 능력

비개발자

"잘 됐다" — 눈으로만 확인

개발자

"얼마나 잘 됐나" 수치화 · 자동 검증

AI 출력은 확률적 — Harness가 없으면 품질 보장 불가



궁극적 차이

서비스를 만드는 것 vs 서비스를 책임지는 것

서비스를 만드는 것

AI가 만들어준 결과물에
의존하는 사람

VS

서비스를 책임지는 것

AI를 도구로 삼아 시스템을
소유(own)하고 통제하는 사람

규모가 커지는 순간 드러나는 차이

트래픽 급증

보안 이슈

비용 폭발

특정 사용자 버그

장애 대응

AI는 진입 장벽을 낮추었지만, 천장은 그대로다

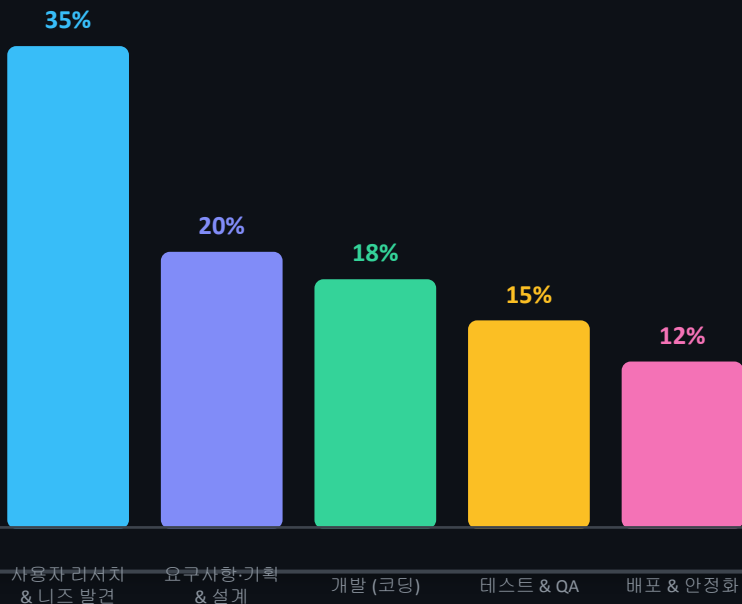
Prompt Engineering을 잘 하는 비개발자 + Harness Engineering을 아는 개발자 = 가장 강력한 조합

개발자는 AI로 더 높은 레이어를 다룰 수 있게 됐고 — 표면에서의 격차는 줄었지만, 깊이에서의 격차는 오히려 벌어졌다

제품 Lifetime에서 가장 많은 시간이 드는 것 — 사용자 이해

제품은 만드는 사람이 아닌, 사용하는 사람 중심이어야 한다

제품 개발 Lifecycle 시간 배분 (대략적 추정)



📊 근거 자료

- NASA 연구: 요구사항에 8~14% 투자한 프로젝트는 비용 초과가 크게 감소; 5% 미만 투자 시 80~200% 초과
- Forrester: UX 리서치에 \$1 투자 시 ROI \$100 (약 9,900%)
- McKinsey: IT 프로젝트의 45%가 예산 초과, 평균 기대효과 56% 미달 — 다수가 니즈 불일치에서 시작
- 스타트업 실패 원인 2위: "시장이 원치 않는 제품" (CB Insights)

💡 AI 시대에도 변하지 않는 것

AI가 코드를 빠르게 만들어줘도, 사용자가 그 변화를 받아들이고 신뢰하는 데는 여전히 시간이 필요합니다. 성패를 가르는 것은 기술이 아니라 사용자에 대한 이해입니다.

👤 이 영역은 지금도, 앞으로도 대부분 사람의 판단 영역입니다

AI 개발 패러다임의 진짜 모습

패러다임은 분명 격변하고 있지만, '현실적인 대부분의 경우'는 이상과 다르다

제품 개발 과정 — 이상 vs 현실 (예시)

✨ 이상

프롬프트 → 완성된 서비스

"며칠 안에 다 끝난다"

✓ 현실

Demo

사용자 리서치 · 요구사항

설계 · 구현

QA · 보안 · 운영
준비

사용자 검증 · 반복

"Demo는 시작일 뿐, 본 게임은 그 이후"

● 예전과 크게 달라진 부분

- 초기 코드·UI 생성 속도 (수주 → 수시간)
- 탐색적 프로토타이핑 비용 거의 0
- 문서·테스트 초안 자동화
- 반복적 패턴 코드(CRUD 등) 자동 처리

● 여전히 사람의 영역인 부분

- 사용자 니즈의 정확한 이해와 정의
- 아키텍처·트레이드오프 판단
- Gradient Quality 평가 (얼마나 좋은가)
- 신뢰 형성 · 변화 수용에 걸리는 시간

패러다임은 바뀌었다. 하지만 "제품의 성패"를 결정하는 기준은 바뀌지 않았다.

AI는 '만드는 속도'를 바꿨다 — 그러나 '무엇을, 누구를 위해 만들 것인가'에 대한 답은 여전히 사람의 몫이다

환상: AI가 모든 것을 끝낸다 | 현실: AI는 시작을 가속하고, 사람은 여전히 방향을 결정한다

Legacy 기업의 AI 도입 — 3가지 접근법

기존 비즈니스 운영과 AI를 통한 새로운 시도 사이의 균형점

 기존 운영 중인 핵심 서비스

vs

 AI를 통한 새로운 시도

1



보조 서비스 (Augment)

추가 Value 제공

- AI 챗봇으로 고객 응대 보조
- AI 생성 콘텐츠로 정보 다양화
- 기존 서비스에 AI 기능 추가

리스크 낮음 · 기존 구조 유지

2



자동화 (Automate)

반복 업무 효율화

- 단순 반복 업무를 AI로 대체
- 운영 비용 절감 · 처리 속도 향상
- 사람은 예외 처리 · 판단에 집중

리스크 중간 · 운영 효율 중심

3



재창조 (Reinvent)

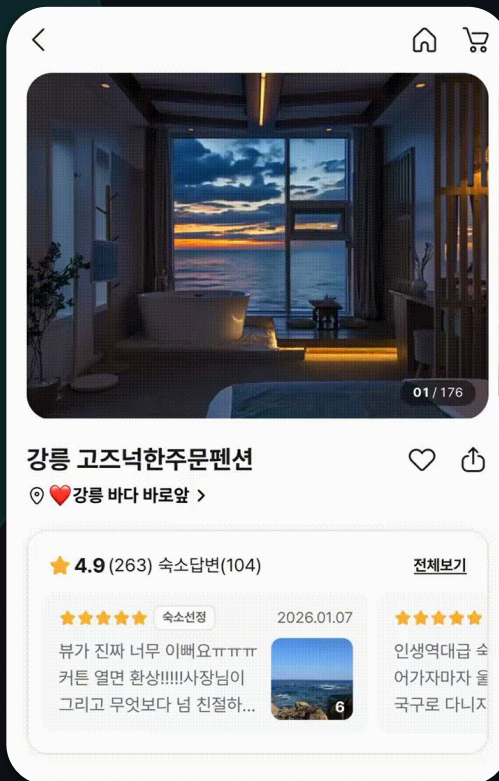
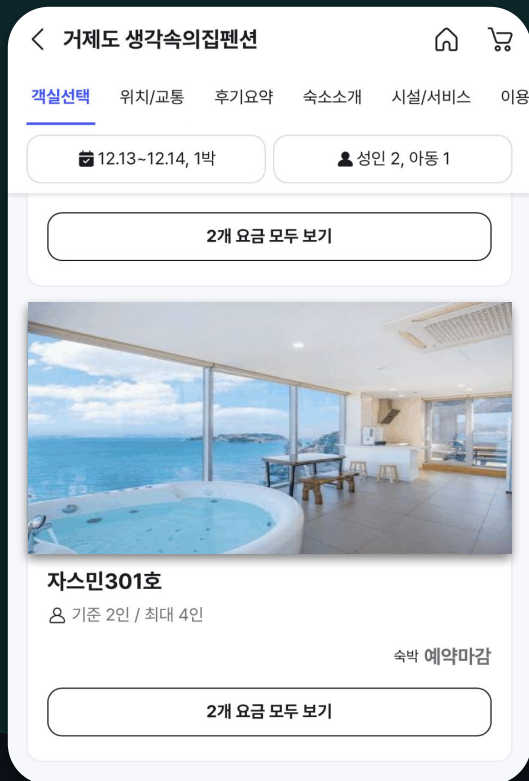
완전히 새로운 가치 창조

- 기존 서비스를 AI-Native로 재정의
- 새로운 가치 제안 (Value Proposition)
- 경쟁 구도 자체를 재편할 잠재력

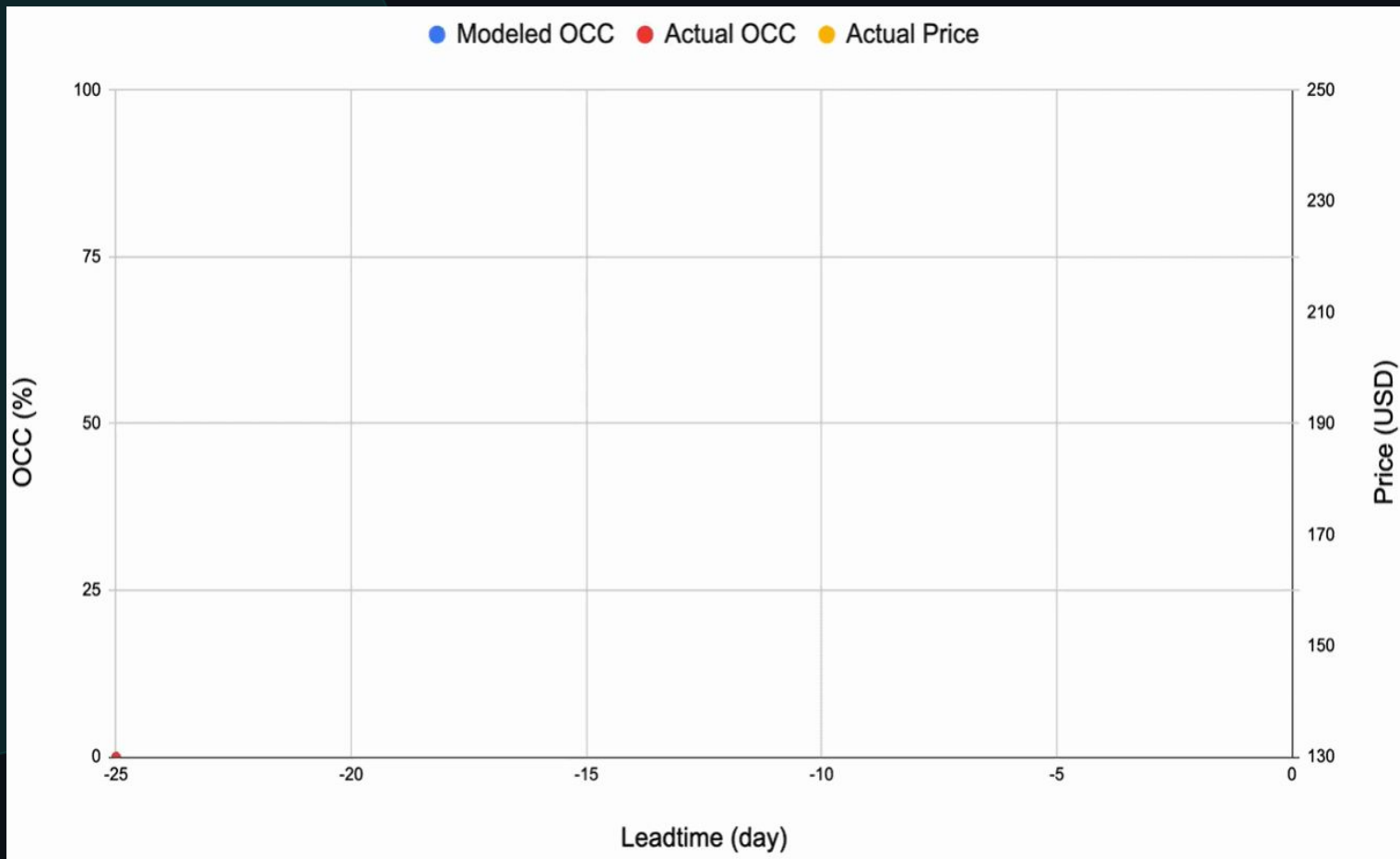
리스크 높음 · 구조적 변화 필요

다음 페이지에서 각 접근법의 구체적인 사례를 살펴봅니다 →

보조 서비스 (Augment) - NOL에서 기존 서비스에 예약율 (Conversion)을 올리기 위한 사용성 개선



자동화 (Automate) - 야놀자의 자동 가격 예측 및 가격 설정 적용



자동화 (Automate) - AI Voice Agent Tella의 아파트 관리 사무소 적용



VOICE AGENT TELLA - IN APARTMENT

자동화 (Automate) - Voice Agent Tella의 Yanolja wholesale 시스템 적용

Tella Intelligence

Auto Live SYSTEM: ACTIVE BRIDGE: LIVE

Booking

AI Calls

Conversation

PMS Verify

HCN

Complete

Korean Japanese Chinese Hindi English

3rd Party

DI Verification

Angry Staff

Start Demo

OTA BOOKING INFORMATION

Source: Stuba



Hilton Cologne

RESERVATION DETAILS

Guest

Barbara Mira

Hotel

Hilton Cologne

Check-in

2026-03-15

Check-out

2026-03-20

Room Type

Queen Guest Room

OTA

Stuba

OTA Ref

STB-9918273

Existing HCN

3318662607

Hotel Confirmation Number

Awaiting



Incoming call from Tella AI

Calling Hilton Cologne

Connecting...

HOTEL PMS

Awaiting call connection

ACTIVITY

AI has securely retrieved the guest reservation from Stuba.

Tella AI is now contacting the hotel front desk via VOIP

재창조 (Reinvent) - Yanolja Hotel Operating System

Housekeeping

Rooms

Generate Tasks

STD-K
222
Clean
No task

FLOOR G

TURNOVER
101
Unassigned

TURNOVER
102
Primary Attendant
Assigned

TURNOVER
103
Second Attendant
Assigned

STAYOVER
202
Primary Attendant
Cleaning

UNSPECIFIED

SUITE
ROH-SUITE-1
Clean
No task

Grand Metropolitan Hotel | Property Dashboard

Admin | Sign out

Hotel OS
Management System

OPERATIONS

Reservations

Calendar

Search

Activity

Availability

Rooms

Profiles

Folios

Night Audit

Settings

Reservation activity

Today's arrivals, departures, and in-house guests.

Arrivals Upcoming Departures In-house Checked-out

Guest	Room type	Room	Cleanliness	Status	Check-in date	Check-out date	Actions
Alice Johnson	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-16	Check in
Dahlia Wolf	DLX-T	—	—	CONFIRMED	2026-05-13	2026-05-14	Check in
Marjorie Konopelski	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-15	Check in
Gaetano Pacocha	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-16	Check in
Catherine Schuster	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-14	Check in
Ulices Jacobi	DLX-T	—	—	CONFIRMED	2026-05-13	2026-05-15	Check in
Cicero Shields	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-16	Check in
Mateo Corwin-Denesik	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-14	Check in
Stephan Dicki	SUITE	—	—	CONFIRMED	2026-05-13	2026-05-15	Check in
Adolphus Langosh	DLX-T	—	—	CONFIRMED	2026-05-13	2026-05-16	Check in
Hans Russel	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-14	Check in
Lois Abernathy	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-15	Check in
Missouri Shields	DLX-T	—	—	CONFIRMED	2026-05-13	2026-05-14	Check in
Maximus Maggio	STD-K	—	—	CONFIRMED	2026-05-13	2026-05-15	Check in
Theresa Bink	SUITE	—	—	CONFIRMED	2026-05-13	2026-05-16	Check in

localhost:7001/reservations/activity

① 체크아웃 (Check-out): 퇴실 처리 → 객실 자동 'dirty' 전환

Quality의 본질

0 / 1 이 아닌 것들

"기능이 있다 없다"가 아닌, "사용자에게 얼마나 좋은가"의 세계

0 / 1 Functionality

됩니다 / 안 됩니다



Gradient Quality

사용하는 주체인 사용자에게 얼마나 좋은가

왜 Gradient Quality 영역엔 사람이 계속 필요한가

Search · Recommendation · Personalization · Localization · Content Moderation



Ground Truth가 없다

①

"이 검색 결과가 더 좋다"는 객관적 정답이 존재하지 않는다.
AI가 학습할 정답 데이터 자체를 사람이 만들고 검증해야 한다.



평가 기준은 사람이 설계한다

②

AI로 품질을 평가해도 "무엇이 좋은 검색인가"의 기준은
사람이 정의한다.
Personalization vs Filter Bubble — 가치 판단.



문화 · 맥락 · 시대가 변한다

③

한국 사용자의 "맛집" 기대와
미국 사용자의 "restaurant" 기대는 다르다.
이 기대는 유행·세대·사회 이슈로 끊임없이 변한다.



메타 평가의 무한 귀납

④

AI 평가 모델의 품질은 누가 판단하는가?
그 메타 평가의 메타 평가는?
결국 사람에서 멈춘다.

앞으로 어떻게 봐야 하는가

Gradient Quality 영역에서 사람 개발자의 역할 변화

Role Shift

관점 ①

소멸이 아닌 역할의 이동

단순 반복 판단은 AI가 대체

기준 설계 · AI 검증 · 예외 처리 ·

새 품질 차원 발견

→ 사람 역할이 상위 레이어로 이동

Permanent Architecture

관점 ②

Human-in-the-Loop는 임시가 아닌 지속 가능한 아키텍처

Search · Recommendation
Content Moderation · Personalization

→ AI 단독 운영으로는 신뢰성 확보 불가
→ 영구적 설계 원칙

New Discipline

관점 ③

Judgment Engineering 새로운 직군 부상

어떤 케이스를 사람이 봐야 하나
어떤 기준으로 판단하나
판단 결과를 AI에 어떻게 피드백하나

→ Search Quality Engineer
Relevance Engineer
Trust & Safety Engineer

AI는 정답 있는 문제를 자동화한다 — 기업의 핵심 경쟁력은 정답 없는 문제에서 나온다 — 그 영역의 사람 개발자는 소멸하지 않고, 더 높은 레이어로 이동한다

CS 학생·연구자를 위한 현실

Demo ≠ Product

관리자와 비개발자가 놓치기 쉬운 것, 학교에서 다루는 것과 다른 관점

Demo

며칠/수주

관리자가 보는 것



Launch

수개월 ~ 수년

Productionize에 실제 걸리는 것

Demo에서 Product까지 — 보이지 않는 90%

관리자가 보는 것 vs 개발자가 실제로 해야 하는 것

Demo를 통해 관리자나 비개발자들이 보는 것

"와, 잘 되네! 언제 출시해요?"

 실제 개발자가 해야 하는 것

 인증 / 인가 / 보안

OAuth, RBAC, 취약점 패치, 침투 테스트

 스케일링 / 가용성

Load balancing, Auto-scaling, 99.9% SLA

 Harness / Eval / QA

회귀 테스트, 품질 게이트, 모니터링 알람

 Localization / 접근성

다국어, WCAG, GDPR 등 법적 요구사항

 비용 최적화

토론 비용, 인프라 비용, 캐싱 전략

 장애 대응 / 롤백

On-call, Incident runbook, 무중단 배포

 림 협업 / 코드 품질

코드 리뷰, 문서화, 기술 부채 관리

 데이터 파이프라인 / 로깅

이벤트 추적, 분석, A/B 테스트 인프라

Demo (시작점)



아이디어 Seed

Production 준비도



100%

≠

Demo는 아이디어의 가장 이상적인 시나리오를 시각화한 시작점

Product화까지: 검증 → 설계 → 구현 → 테스트 → 운영

이 여정에 뛰어난 개발자가 필요하다

학교와 현업 사이, 알아두면 좋은 5가지 현실

아카데미에서의 경험 vs 기업 현장에서 요구되는 것

① 완벽 vs 출시

알고리즘 · 자료구조 / 최적 솔루션을 찾아라

→ **Good Enough + 빠른 출시 / 완벽한 설계보다 작동하는 것이 먼저**

② 문제 정의

문제가 주어진다 / 정답이 존재한다

→ 문제를 정의하는 것이 일의 절반 / 무엇을 만들어야 하는지 모를 때도 많다

③ 협업과 소통

혼자 또는 소수팀 / 코드만 잘 짜면 된다

→ **PM · 디자이너 · 데이터 · 비즈니스와 협업 / 커뮤니케이션이 기술만큼 중요하다**

④ 운영·유지보수

논문 한 편의 실험 / 결과만 좋으면 된다

→ **운영 · 모니터링 · 패치 · 보안 / 배포 후가 진짜 시작이다**

⑤ 기술 부채

기술 부채? 일단 제출 / 다음 학기엔 새 과제

→ **기술 부채는 누적되고 폭발한다 / 리팩토링 · 문서화 · 테스트가 생존 전략**

지금 당신이 가져야 할 관점

CS 학생 · 대학원생 · 연구자에게

▲ T자형 개발자가 되어라

↑ ↓ 깊이 (Depth)

한 분야를 세상에서 가장 깊이 파라
그것이 당신만의 무기가 된다

↔ 넓이 (Breadth)

보안 · 운영 · 협업 · 비즈니스 맥락
팀의 언어를 읽을 수 있어야 한다

🔗 연결 (Bridge)

기술과 비즈니스 사이를 번역하는
개발자가 가장 희소하고 가장 강하다



지금 당장 할 수 있는 것

1

사이드 프로젝트를 실제 배포까지 해보라

Github Pages라도 좋다. 운영의 무게를 느껴라

2

코드 리뷰 문화에 뛰어들어라

오픈소스 PR, 동료 코드 읽기 — 팀 개발을 배워라

3

장애 리포트 (Post-mortem)를 읽어라

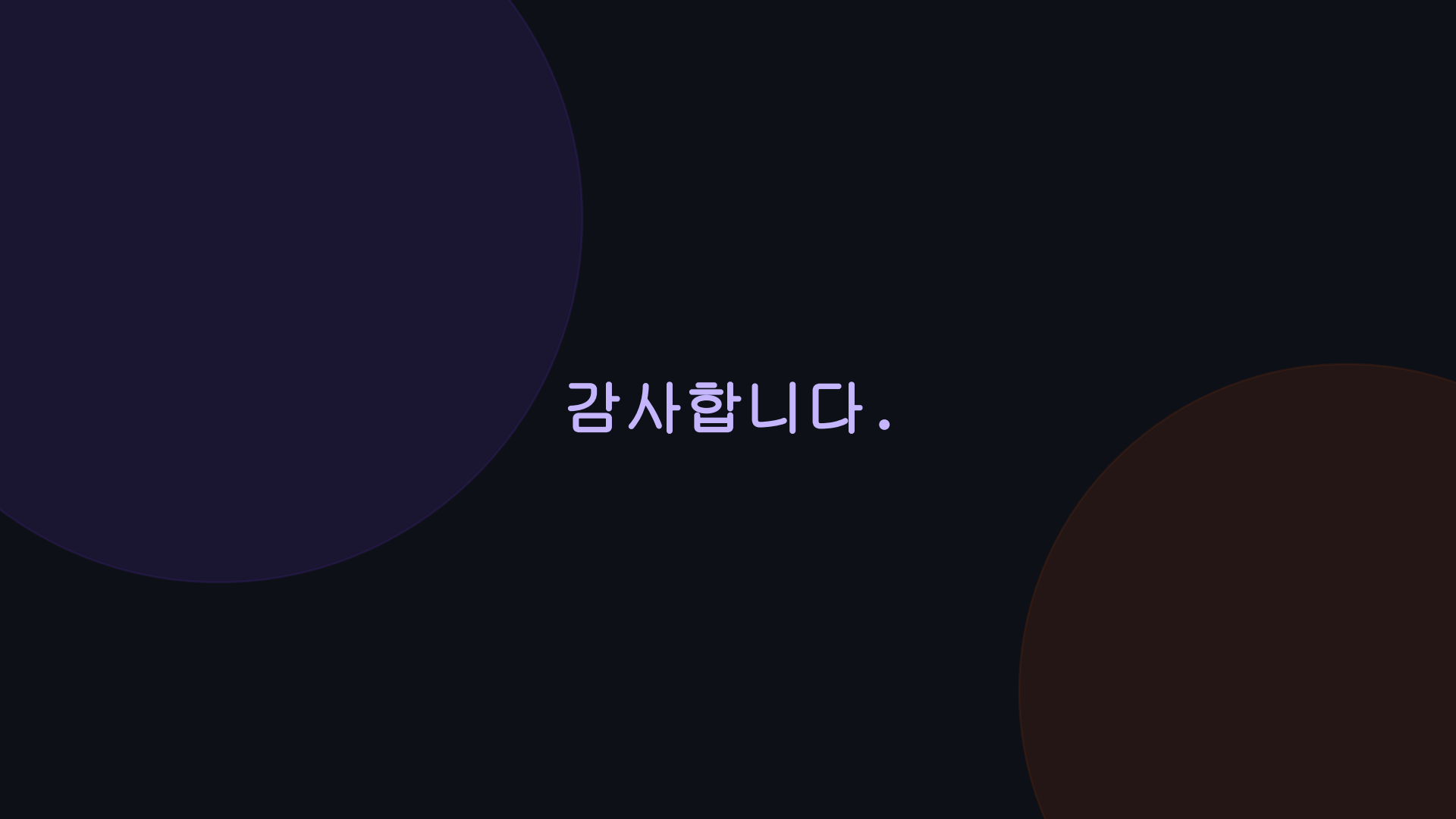
Google · Stripe · AWS 사고 보고서는 최고의 실전 교재

4

비즈니스 지표를 공부하라

DAU · LTV · CAC · Churn — 숫자로 말할 수 있어야 한다

학교는 생각하는 법을 가르쳤다 — 기업은 그 생각으로 세상을 바꾸길 기대한다 — 그 사이의 거리를 지금부터 좁혀라



감사합니다.